Analyze the trend of data

In [1]:
```
!pip install matplotlib
```

```
Requirement already satisfied: matplotlib in /opt/miniconda3/lib/python3.14/
site-packages (3.11.1)
Requirement already satisfied: contourpy>=1.0.1 in /opt/miniconda3/lib/pytho
n3.14/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /opt/miniconda3/lib/python3.1
4/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /opt/miniconda3/lib/pyth
on3.14/site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/miniconda3/lib/pyth
on3.14/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in /opt/miniconda3/lib/python3.1
4/site-packages (from matplotlib) (2.5.2)
Requirement already satisfied: packaging>=20.0 in /opt/miniconda3/lib/python
3.14/site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=9 in /opt/miniconda3/lib/python3.14/s
ite-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /opt/miniconda3/lib/python3.1
4/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/miniconda3/lib/p
ython3.14/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/miniconda3/lib/python3.14/si
te-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [3]:
```
data = {
    "Year": [2015, 2016, 2017, 2018, 2019,
             2020, 2021, 2022, 2023, 2024],

    "Job_Postings": [1200, 1800, 2500, 3500, 4800,
                     6200, 8000, 9500, 10500, 12000]
}

df = pd.DataFrame(data)
```

In [4]:
```
print("Basic Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nYear with highest job postings:")
print(df.loc[df["Job_Postings"].idxmax()])
```

```
Basic Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Year          10 non-null     int64
 1   Job_Postings  10 non-null     int64
dtypes: int64(2)
memory usage: 292.0 bytes
None

Statistical Summary:
              Year  Job_Postings
count    10.00000      10.00000
mean   2019.50000    6000.00000
std       3.02765    3846.49913
min    2015.00000    1200.00000
25%    2017.25000    2750.00000
50%    2019.50000    5500.00000
75%    2021.75000    9125.00000
max    2024.00000   12000.00000

Year with highest job postings:
Year             2024
Job_Postings    12000
Name: 9, dtype: int64
```

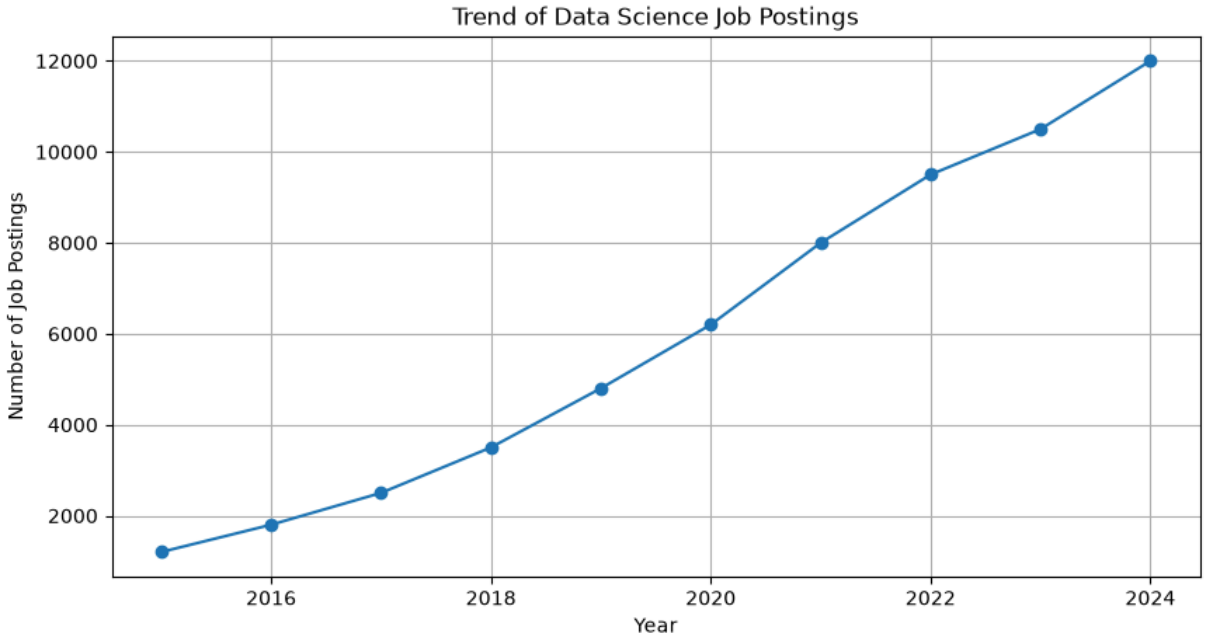
In [5]:

```python
plt.figure(figsize=(10, 5))

plt.plot(df["Year"], df["Job_Postings"], marker="o")

plt.title("Trend of Data Science Job Postings")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.grid(True)

plt.show()
```



In [6]:
```
roles = {
    "Role": [
        "Data Scientist",
        "Data Analyst",
        "Data Engineer",
        "ML Engineer",
        "Business Analyst"
    ],

    "Count": [350, 280, 220, 180, 120]
}

role_df = pd.DataFrame(roles)

role_df
```

Out[6]:

	Role	Count
0	Data Scientist	350
1	Data Analyst	280
2	Data Engineer	220
3	ML Engineer	180
4	Business Analyst	120

In [7]:
```
print("Role Distribution:")
print(role_df)

print("\nMost common role:")
print(role_df.loc[role_df["Count"].idxmax()])
```

```
Role Distribution:
                Role  Count
0    Data Scientist    350
1      Data Analyst    280
2     Data Engineer    220
3       ML Engineer    180
4  Business Analyst    120

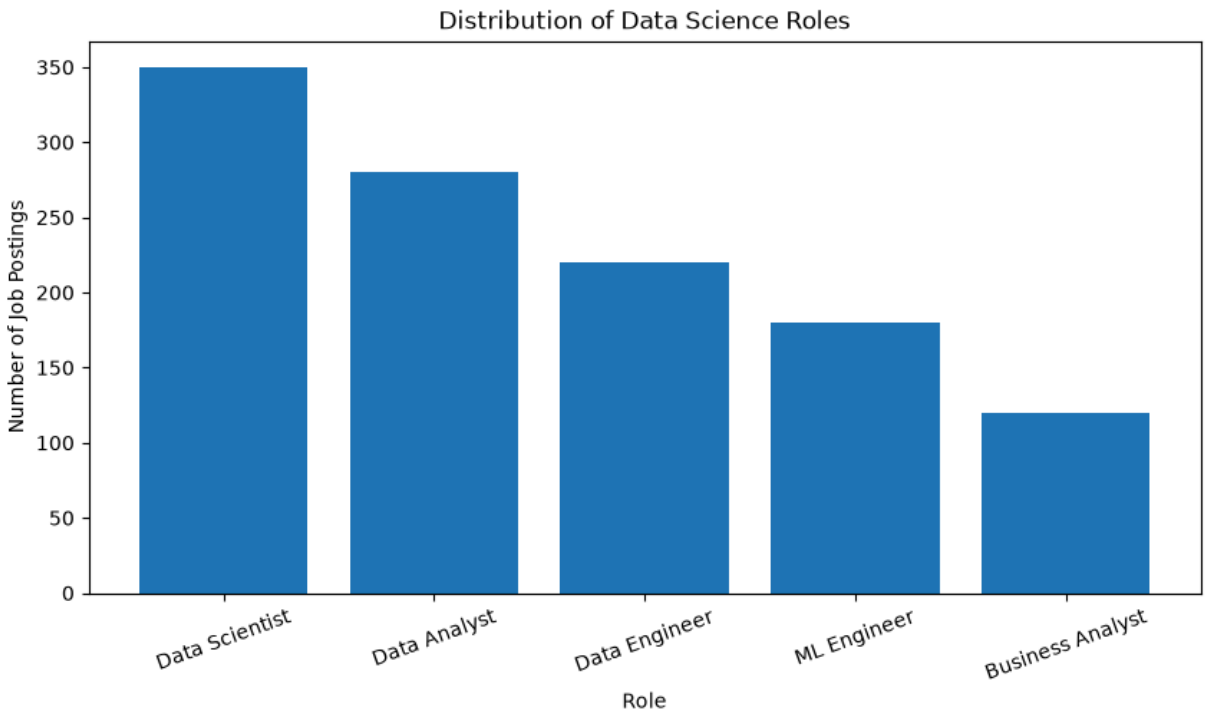
Most common role:
Role     Data Scientist
Count               350
Name: 0, dtype: object
```

In [8]:
```python
plt.figure(figsize=(10, 5))

plt.bar(role_df["Role"], role_df["Count"])

plt.title("Distribution of Data Science Roles")
plt.xlabel("Role")
plt.ylabel("Number of Job Postings")
plt.xticks(rotation=20)

plt.show()
```



In [9]:
```python
import pandas as pd
import json
```

In [10]:
```python
structured_data = {
    "ID": [1, 2, 3],
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [22, 24, 21]
}

structured_df = pd.DataFrame(structured_data)
```

```
print("STRUCTURED DATA")
print(structured_df)
```

```
STRUCTURED DATA
   ID     Name  Age
0   1    Alice   22
1   2      Bob   24
2   3  Charlie   21
```

In [11]:
```
semi_structured_data = {
    "students": [
        {
            "ID": 1,
            "Name": "Alice",
            "Skills": ["Python", "SQL"]
        },
        {
            "ID": 2,
            "Name": "Bob",
            "Skills": ["Java", "C++"]
        }
    ]
}

print("SEMI-STRUCTURED DATA")
print(json.dumps(semi_structured_data, indent=4))
```

```
SEMI-STRUCTURED DATA
{
    "students": [
        {
            "ID": 1,
            "Name": "Alice",
            "Skills": [
                "Python",
                "SQL"
            ]
        },
        {
            "ID": 2,
            "Name": "Bob",
            "Skills": [
                "Java",
                "C++"
            ]
        }
    ]
}
```

In [12]:
```
unstructured_data = """
Data Science is an interdisciplinary field.
It combines statistics, machine learning,
programming and data visualization.
"""
```

```
print("UNSTRUCTURED DATA")
print(unstructured_data)
```

```
UNSTRUCTURED DATA

Data Science is an interdisciplinary field.
It combines statistics, machine learning,
programming and data visualization.
```

In [13]:
```
comparison = pd.DataFrame({
    "Data Type": [
        "Structured",
        "Semi-Structured",
        "Unstructured"
    ],

    "Example": [
        "SQL Table / Excel",
        "JSON / XML",
        "Text / Images / Videos"
    ]
})

comparison
```

Out[13]:

	Data Type	Example
0	Structured	SQL Table / Excel
1	Semi-Structured	JSON / XML
2	Unstructured	Text / Images / Videos

In [15]:
```
from cryptography.fernet import Fernet
```

In [16]:
```
key = Fernet.generate_key()

print("Encryption Key:")
print(key)
```

```
Encryption Key:
b'tGou_PQFx3GwOXfUOixJrfwNWwZyeoKC1JOYJFfYCKQ='
```

In [17]:
```
cipher = Fernet(key)

message = "My sensitive data"

encrypted_message = cipher.encrypt(message.encode())

print("Original Data:")
print(message)

print("\nEncrypted Data:")
print(encrypted_message)
```

```
Original Data:
My sensitive data

Encrypted Data:
b'gAAAAABqhZ7kbVRDjvhaRTOE_77Yrk7QX5Yh92hNwq0hsfT51pZ8KZEC9dND0efGFz4vcqtMF7
RWLeOVs1BQPLWuchQjd6yTXk-zuJ2e5n2DDHrv1d4Pnoc='
```

In [18]:
```
decrypted_message = cipher.decrypt(encrypted_message).decode()

print("Decrypted Data:")
print(decrypted_message)
```

```
Decrypted Data:
My sensitive data
```

In [ ]:
```

```